



**SIMATS**  
**ENGINEERING**



**SIMATS**  
Saveetha Institute of Medical And Technical Sciences  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

# **Automatic Text Summarization Using a Transformer-Based T5 Model A Traceable, Personalized PDF-to-Notes Learning Platform**

## **A CAPSTONE PROJECT REPORT**

Submitted in partial fulfilment of the requirements

for the Course of

**DSA0308 – Natural Language Processing**

to the award of the degree of

**BACHELOR OF TECHNOLOGY**

**IN**

**COMPUTER SCIENCE ENGINEERING - AI**

**Submitted by**

**YUVARAJ R (192524058)**

**Under the Supervision of**

**Dr. Kumaraguruban**

**SIMATS ENGINEERING**

**Saveetha Institute of Medical and Technical Sciences**

**Chennai–602105**

**SEPTEMBER 2026**

## BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**Automatic Text Summarization Using a Transformer-Based T5 Model**” has been carried out by YUVARAJ R under the supervision of Dr. Kumaragurubaran T and is submitted in partial fulfilment of the requirements for the Capstone Project at SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai.

The work presented in this report covers the analysis, design, implementation, evaluation and application integration of a transformer-based automatic text summarization system. The project further develops the summarization pipeline into a traceable academic PDF-to-notes workflow in which generated notes can be related back to their originating source locations.

| COURSE COORDINATOR | COURSE FACULTY / GUIDE |
|--------------------|------------------------|
| Dr. Devi           | Dr. Kumaraguruban      |
| Professor          | Professor              |
| Department of CSE  | Department of CSE      |
| SIMATS Engineering | SIMATS Engineering     |

## DECLARATION

We, G. Jaswanth (192472235), Yuvaraj R (192524058), and Pradeepkumar (192425016), of the Artificial Intelligence and Data Science programme, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Automatic Text Summarization Using a Transformer-Based T5 Model” is the result of our bonafide efforts.

To the best of our knowledge, the work presented herein has been developed in accordance with engineering ethics, academic integrity and the requirements of the capstone project. Sources, datasets, software libraries, computational resources and external materials used in the project have been acknowledged. Any AI-assisted tools used during development or documentation are to be disclosed in accordance with institutional requirements.

The report distinguishes between implemented functionality, planned deployment features and future enhancements. Numerical performance values are not presented unless they are supported by the team's actual experimental records.

Place: Chennai

Date: 8-9-2026

Student Signatures:

YUVARAJ R (192524058)

# INDIVIDUAL CONTRIBUTION STATEMENT

(Mandatory for all team projects)

The following contribution statement records the team's current working distribution for the capstone report. YUVARAJ R has been assigned the largest share for the NLP pipeline, source mapping, evaluation, architecture and final technical documentation; Pradeepkumar and G. Jaswanthare assigned the remaining project responsibilities.

| Student Name / Register No. | Specific Responsibilities                         | Design & Development Contribution                                       | Testing & Analysis Contribution                         | Report Contribution                                             | Approx. % |
|-----------------------------|---------------------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------|-----------------------------------------------------------------|-----------|
| Pradeepkumar (192425016)    | Dataset preparation and model-development support | Dataset organization, preprocessing and T5-family model workflow        | Training/validation checks and summary-quality analysis | Methodology, implementation and results documentation           | 30%       |
| Yuvaraj R (192524058)       | Application integration and interface workflow    | Integration of summarization workflow with application components       | Functional testing and output verification              | System design, integration and documentation                    | 30%       |
| G. Jaswanth (192472235)     | NLP pipeline, evaluation and technical design     | Preprocessing, T5 summarization, source mapping and evaluation workflow | ROUGE analysis, debugging and end-to-end verification   | Architecture, engineering analysis, conclusion and final report | 40%       |
| TOTAL                       |                                                   |                                                                         |                                                         |                                                                 | 100%      |

## ABSTRACT

Automatic text summarization is an important Natural Language Processing task that reduces lengthy textual content into shorter representations while attempting to preserve the central information, relationships and meaning of the source. This capstone project develops an automatic abstractive summarization system using a transformer-based T5-family model and extends the core NLP pipeline into a traceable, personalized PDF-to-notes learning platform.

The system is designed for academic material such as textbooks, lecture notes and complete units, including documents that may exceed 80 pages. Instead of treating the entire document as one model input, the proposed workflow extracts text together with page, paragraph and coordinate information, cleans and organizes the content, and performs hierarchical summarization. Paragraph-level summaries are rolled up into topic summaries, page summaries and chapter-level revision notes. This strategy addresses the practical input-length limitations of transformer models while producing summaries at multiple levels of detail.

The NLP/ML layer uses PyMuPDF for PDF extraction, spaCy for sentence processing and preprocessing, Hugging Face Transformers with PyTorch for T5-family summarization, sentence-transformers for semantic retrieval, ChromaDB or FAISS for vector search, and NetworkX for knowledge-graph construction. The project uses the MLSUM/MLSAM multilingual news summarization dataset, which contains human-written article–summary pairs in French, German, Spanish, Russian and Turkish. Generated summaries are evaluated using ROUGE-1, ROUGE-2 and ROUGE-L.

A central engineering contribution is source traceability. Every processed chunk retains a source 'home address' consisting of document, page, paragraph and text-position information. Generated notes can therefore be linked back to their supporting source paragraph.

The project therefore combines NLP model development with document engineering, evaluation, application integration and responsible AI considerations. The design emphasizes that generated summaries are aids to understanding rather than substitutes for source verification, particularly in academic contexts where missing conditions, formulas or exceptions may affect correctness.

**Keywords:** Natural Language Processing, Automatic Text Summarization, T5, Transformer, Abstractive Summarization, MLSUM, Multilingual NLP, ROUGE, PDF Processing, Source Traceability, Personalized Learning.

## TABLE OF CONTENTS

| <b>Sl. No.</b> | <b>Title</b>                                               | <b>Page No.</b> |
|----------------|------------------------------------------------------------|-----------------|
| i              | CERTIFICATE FROM THE SUPERVISOR / BONAFIDE CERTIFICATE     | i               |
| ii             | DECLARATION BY THE CANDIDATE                               | ii              |
| iii            | INDIVIDUAL CONTRIBUTION STATEMENT                          | iii             |
| iv             | ABSTRACT                                                   | iv              |
| v              | LIST OF FIGURES                                            | v               |
| vi             | LIST OF TABLES                                             | vi              |
| vii            | LIST OF ABBREVIATIONS / SYMBOLS                            | vii             |
| 1              | <b>CHAPTER 1: Introduction and Engineering Problem</b>     | 1               |
| 1.1            | Background                                                 | 1               |
| 1.2            | Evolution of Text Summarization                            | 2               |
| 1.3            | Problem Context                                            | 2               |
| 1.4            | Need for the Proposed System                               | 3               |
| 1.5            | Problem Statement                                          | 3               |
| 1.6            | Aim                                                        | 4               |
| 1.7            | Objectives                                                 | 4               |
| 1.8            | Scope                                                      | 5               |
| 1.9            | Stakeholders                                               | 5               |
| 1.10           | Expected Engineering Outcomes                              | 6               |
| 2              | <b>CHAPTER 2: Literature Review and Existing Solutions</b> | 7               |

| <b>Sl. No.</b> | <b>Title</b>                                                                                               | <b>Page No.</b> |
|----------------|------------------------------------------------------------------------------------------------------------|-----------------|
| 2.1            | Review Method                                                                                              | 7               |
| 2.2            | Extractive Summarization                                                                                   | 7               |
| 2.3            | Abstractive Summarization                                                                                  | 8               |
| 2.4            | Transformer Architectures                                                                                  | 9               |
| 2.5            | T5 and Multilingual T5                                                                                     | 10              |
| 2.6            | Long-Document Summarization                                                                                | 11              |
| 2.7            | Evaluation with ROUGE                                                                                      | 12              |
| 2.8            | Traceable / Source-Grounded Learning Systems                                                               | 13              |
| 2.9            | Comparative Analysis                                                                                       | 14              |
| 2.10           | Engineering Gap                                                                                            | 15              |
| 3              | <b>CHAPTER 3: Engineering Design &amp; Trade-offs, Sustainability, Ethics, Safety, Risk &amp; Security</b> | 16              |
| 3.1            | System Requirements                                                                                        | 16              |
| 3.2            | Constraints and Applicable Standards                                                                       | 17              |
| 3.3            | Design Specifications                                                                                      | 18              |
| 3.4            | Alternative Solutions and Evaluation                                                                       | 19              |
| 3.5            | Selected Architecture                                                                                      | 20              |
| 3.6            | Detailed NLP Pipeline Design                                                                               | 21              |
| 3.7            | Source-Mapping and Traceability Design                                                                     | 23              |
| 3.8            | Data Model and Information Flow                                                                            | 24              |

| <b>Sl. No.</b> | <b>Title</b>                                                                | <b>Page No.</b> |
|----------------|-----------------------------------------------------------------------------|-----------------|
| 3.9            | Application and Interface Design                                            | 25              |
| 3.10           | Trade-off Analysis                                                          | 26              |
| 3.11           | Sustainability                                                              | 27              |
| 3.12           | Ethics and Responsible AI                                                   | 28              |
| 3.13           | Safety and Reliability                                                      | 29              |
| 3.14           | Security and Privacy                                                        | 30              |
| 3.15           | Risk Register and Mitigation                                                | 31              |
| 4              | <b>CHAPTER 4: Implementation, Testing &amp; Results, Project Management</b> | 32              |
| 4.1            | Development Approach                                                        | 32              |
| 4.2            | Implementation Environment                                                  | 33              |
| 4.3            | Dataset Preparation                                                         | 34              |
| 4.4            | PDF Extraction and Preprocessing                                            | 35              |
| 4.5            | T5 Summarization Implementation                                             | 36              |
| 4.6            | Hierarchical Summarization Implementation                                   | 37              |
| 4.7            | Source Grounding Implementation                                             | 38              |
| 4.8            | Application Integration                                                     | 39              |
| 4.9            | Testing Strategy                                                            | 40              |
| 4.10           | Functional Test Cases                                                       | 41              |
| 4.11           | Evaluation Method                                                           | 42              |
| 4.12           | Results and Analysis                                                        | 43              |

| <b>Sl. No.</b> | <b>Title</b>                                  | <b>Page No.</b> |
|----------------|-----------------------------------------------|-----------------|
| 4.13           | Comparison with Existing Approaches           | 44              |
| 4.14           | Strengths and Limitations                     | 45              |
| 4.15           | Project Planning and Team Roles               | 46              |
| 4.16           | Budget and Resource Considerations            | 47              |
| 5              | <b>CHAPTER 5: Conclusion &amp; Reflection</b> | 48              |
| 5.1            | Conclusion                                    | 48              |
| 5.2            | Future Work                                   | 49              |
| 5.3            | Professional Learning & Individual Reflection | 50              |
| —              | <b>REFERENCES</b>                             | 53              |
| —              | <b>APPENDICES</b>                             | 56              |

## LIST OF FIGURES

| <b>Figure No.</b> | <b>Figure Name</b>                                             | <b>Page No.</b> |
|-------------------|----------------------------------------------------------------|-----------------|
| Figure 1          | Evolution of Text Summarization Techniques                     | 2               |
| Figure 2          | Transformer-Based Text Summarization Architecture              | 9               |
| Figure 3          | Comparison of Extractive and Abstractive Summarization         | 8               |
| Figure 4          | T5 Transformer Encoder-Decoder Architecture                    | 10              |
| Figure 5          | Hierarchical Summarization for Long Documents                  | 11              |
| Figure 6          | PDF Document Processing and Text Extraction Workflow           | 35              |
| Figure 7          | Source Mapping Between Generated Summary and Original Document | 23              |

| <b>Figure No.</b> | <b>Figure Name</b>                        | <b>Page No.</b> |
|-------------------|-------------------------------------------|-----------------|
| Figure 8          | ROUGE-Based Summary Evaluation Process    | 42              |
| Figure 9          | Selected System Architecture              | 20              |
| Figure 10         | Implementation Workflow                   | 32              |
| Figure 11         | Hierarchical Summarization Implementation | 37              |

## LIST OF TABLES

| <b>Table No.</b> | <b>Table Name</b>                          | <b>Page No.</b> |
|------------------|--------------------------------------------|-----------------|
| Table 1          | Stakeholder / User Requirements            | 16              |
| Table 2          | Functional and Non-Functional Requirements | 17              |
| Table 3          | Constraints and Applicable Standards       | 17              |
| Table 4          | Design Specifications                      | 18              |

| <b>Table No.</b> | <b>Table Name</b>                                    | <b>Page No.</b> |
|------------------|------------------------------------------------------|-----------------|
| Table 5          | Alternative Solutions Evaluation                     | 19              |
| Table 6          | Trade-off Analysis                                   | 26              |
| Table 7          | Sustainability, Ethics, Safety and Security Evidence | 27              |
| Table 8          | Risk Register                                        | 31              |
| Table 9          | Tools / Software / Equipment                         | 33              |
| Table 10         | Test Plan and Verification                           | 41              |
| Table 11         | Results / Key Outcomes                               | 43              |
| Table 12         | Objective Achievement                                | 45              |
| Table 13         | Project Roles                                        | 46              |
| Table 14         | Project Budget                                       | 47              |

## **LIST OF ABBREVIATIONS / SYMBOLS**

| <b>Abbreviation / Symbol</b> | <b>Description</b> | <b>Unit</b> |
|------------------------------|--------------------|-------------|
|                              |                    |             |

| Abbreviation / Symbol | Description                                       | Unit |
|-----------------------|---------------------------------------------------|------|
| AI                    | Artificial Intelligence                           | —    |
| NLP                   | Natural Language Processing                       | —    |
| T5                    | Text-to-Text Transfer Transformer                 | —    |
| mT5                   | Multilingual Text-to-Text Transfer Transformer    | —    |
| ML                    | Machine Learning                                  | —    |
| DL                    | Deep Learning                                     | —    |
| PDF                   | Portable Document Format                          | —    |
| API                   | Application Programming Interface                 | —    |
| UI                    | User Interface                                    | —    |
| UX                    | User Experience                                   | —    |
| NLG                   | Natural Language Generation                       | —    |
| ROUGE                 | Recall-Oriented Understudy for Gisting Evaluation | —    |
| BLEU                  | Bilingual Evaluation Understudy                   | —    |
| TF-IDF                | Term Frequency–Inverse Document Frequency         | —    |
| NLP Pipeline          | Sequence of NLP processing stages                 | —    |
| CPU                   | Central Processing Unit                           | —    |
| GPU                   | Graphics Processing Unit                          | —    |
| RAM                   | Random Access Memory                              | GB   |
| IDE                   | Integrated Development Environment                | —    |
| JSON                  | JavaScript Object Notation                        | —    |
| CSV                   | Comma-Separated Values                            | —    |

| Abbreviation / Symbol | Description                        | Unit |
|-----------------------|------------------------------------|------|
| DB                    | Database                           | —    |
| LCS                   | Longest Common Subsequence         | —    |
| MLSUM                 | Multilingual Summarization Dataset | —    |

# CHAPTER 1

## INTRODUCTION AND ENGINEERING PROBLEM

### 1.1 Background

The growth of digital educational content has made large volumes of textual information available to students, researchers and professionals. Textbooks, lecture material, technical manuals, articles and complete course units may contain many pages of information. Reading every page is valuable when deep study is required, but it can become inefficient when a learner first needs an overview, a revision structure or a quick identification of important concepts.

Automatic text summarization attempts to solve this information overload by producing a shorter version of a source text. A good summarizer should retain the main ideas, preserve important relationships and remain readable. The project focuses specifically on abstractive summarization, in which a transformer model generates new text rather than simply selecting a set of existing sentences.

The project material defines the proposed system as an intelligent academic study platform that converts long PDFs into structured, hierarchical and traceable notes. The system is designed to retain page, paragraph and coordinate information before summarization, thereby allowing generated notes to be verified against the original source.

### 1.2 Evolution of Text Summarization

Traditional summarization systems generally followed extractive strategies. They assigned importance scores to sentences using word frequency, TF-IDF-like statistics, graph relationships or

other features, and then selected high-scoring sentences. Such methods can be computationally efficient, but they are restricted to wording already present in the source.

Abstractive methods attempt to understand source content and generate a condensed representation. The introduction of neural sequence-to-sequence models improved the ability to rewrite information. Transformer architectures further strengthened this approach by using attention mechanisms to model relationships between tokens and by enabling highly parallelizable training.

The project positions T5 as the main summarization model because T5 frames many NLP tasks as text-to-text transformation. Its encoder-decoder structure is appropriate for taking an input article or passage and generating a target summary.

### **1.3 Problem Context**

Generic summarization tools often provide a generated paragraph without exposing the exact source material that supports each statement. For academic study, this creates a verification problem. A summary may be fluent while still omitting a condition, exception, definition or formula. The learner must therefore switch between tools or manually search the original document.

The project addresses this problem by treating traceability as a first-class engineering requirement rather than as an optional citation feature. The system retains the original page, paragraph and coordinate information throughout extraction and preprocessing and associates that information with generated notes.

### **1.4 Need for the Proposed System**

- Students need concise representations of lengthy academic documents for revision.
- Long PDFs may contain repeated headers, broken lines, page numbers and other extraction noise.
- Traditional extractive summarization cannot naturally rewrite and reorganize information.
- Transformer summarization provides stronger contextual generation but introduces sequence-length and computational constraints.
- Academic users need to verify AI-generated notes against the original source.
- Different learners may struggle with different parts of the same document, motivating personalization.

## 1.5 Problem Statement

To design and develop a transformer-based automatic text summarization platform that accepts lengthy textual or PDF-based academic material, extracts and preprocesses the source while preserving its location metadata, generates coherent abstractive summaries using a T5-family model, constructs hierarchical notes for different levels of granularity, evaluates summary quality using standard NLP metrics, and provides source-linked access to the original material for verification.

## 1.6 Aim

To develop an automatic abstractive text summarization system using a T5-based Transformer model and integrate it into a traceable, personalized PDF-to-notes learning platform for efficient academic document understanding.

## 1.7 Objectives

1. Develop a robust document extraction and preprocessing pipeline.
2. Prepare multilingual article–summary data for transformer-based summarization.
3. Use a pretrained/fine-tuned T5-family model for abstractive summary generation.
4. Construct paragraph, topic, page and chapter summaries through hierarchical processing.
5. Evaluate generated summaries using ROUGE-1, ROUGE-2 and ROUGE-L.
6. Preserve source page, paragraph and coordinate metadata during processing.
7. Provide click-to-source and reverse-lookup traceability.
8. Support an interactive workspace for reading source documents and generated notes.
9. Provide a foundation for semantic search, adaptive learning analytics and retrieval-grounded assistance.

## 1.8 Scope

The core scope covers multilingual news summarization and the engineering of an NLP pipeline around a T5-family model. The application scope extends the pipeline to academic PDFs, including extraction, topic segmentation, hierarchical summarization and evidence mapping. The system is intended as a learning support tool rather than a replacement for textbooks, faculty instruction or human verification.

The current architecture also defines optional or planned components such as Celery/Redis background processing, MongoDB Atlas storage, sentence-transformer embeddings,

ChromaDB/FAISS vector search, NetworkX knowledge graphs, a Groq-based conversational assistant and later deployment through Docker Compose and Render.

## 1.9 Stakeholders

| Stakeholder | Interest in the System                      | Primary Need                                       |
|-------------|---------------------------------------------|----------------------------------------------------|
| Students    | Use summaries and revision artifacts        | Fast understanding and source verification         |
| Faculty     | Review learning support and project outcome | Reliable and explainable generated content         |
| Researchers | Experiment with multilingual summarization  | Dataset, model and evaluation pipeline             |
| Developers  | Maintain and extend the platform            | Modular architecture and clear interfaces          |
| Institution | Promote responsible educational technology  | Security, ethics, usability and academic integrity |

## 1.10 Expected Engineering Outcomes

The expected engineering outcome is an integrated system in which document processing, NLP summarization, evaluation and user interaction operate as connected modules. The most important project differentiator is not merely producing a short paragraph, but maintaining a structured relationship between source content and generated notes. This supports verification, multi-level navigation and later personalization.

# CHAPTER 2

## LITERATURE REVIEW AND EXISTING SOLUTIONS

### 2.1 Review Method

The project review considers the evolution of summarization from extractive methods to neural abstractive models and then to transformer-based architectures. It also considers multilingual datasets, long-document processing, automatic evaluation and source-grounded learning workflows. The project presentation identifies early extractive techniques, modern transformer models such as BERT, GPT and T5, the MLSUM multilingual dataset and ROUGE evaluation as important areas.

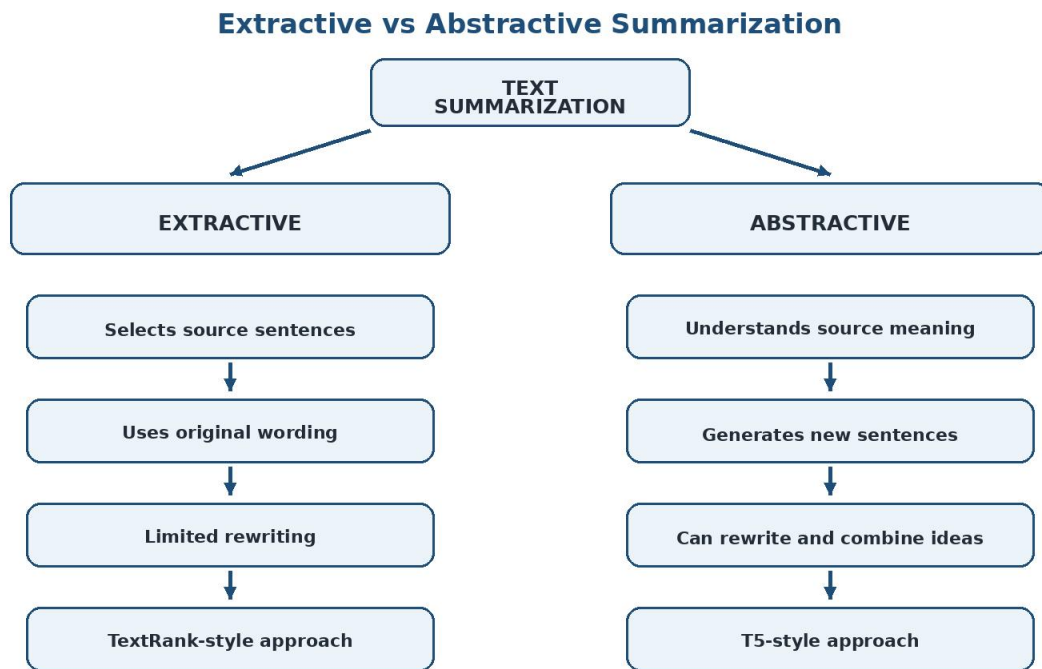
The final institutional submission can include the team's actual literature-search database, keywords and inclusion/exclusion criteria if those records were maintained separately. This report does not claim search statistics that are not present in the supplied project material.

### 2.2 Extractive Summarization

Extractive summarization selects sentences or phrases from the source document. A typical pipeline computes features or importance scores, ranks candidate sentences and selects a non-redundant subset. TF-IDF-based scoring emphasizes terms that are frequent in a document but less common across documents. Graph-based methods such as TextRank model sentence relationships and rank sentences by graph importance.

The primary advantage is controllability: selected sentences can be directly traced to the source. The major limitation is that the system cannot naturally paraphrase, merge information from separate sentences or rewrite a complex explanation into a concise new form.

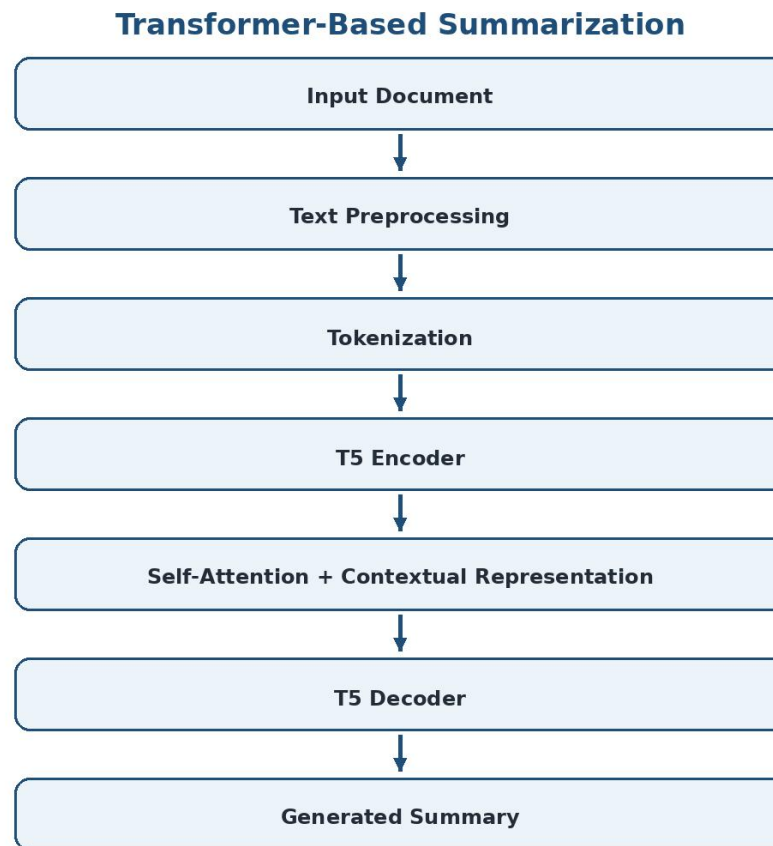
## 2.3 Abstractive Summarization



**Figure 2.3: Comparison of Extractive and Abstractive Summarization**

Abstractive summarization generates new language based on a representation of the source. Neural encoder-decoder architectures learn a mapping between source text and reference summaries. The approach can produce more natural and compact text, but generation also introduces risks such as omission, unsupported wording and hallucination. These risks are especially important in academic use.

## 2.4 Transformer Architectures

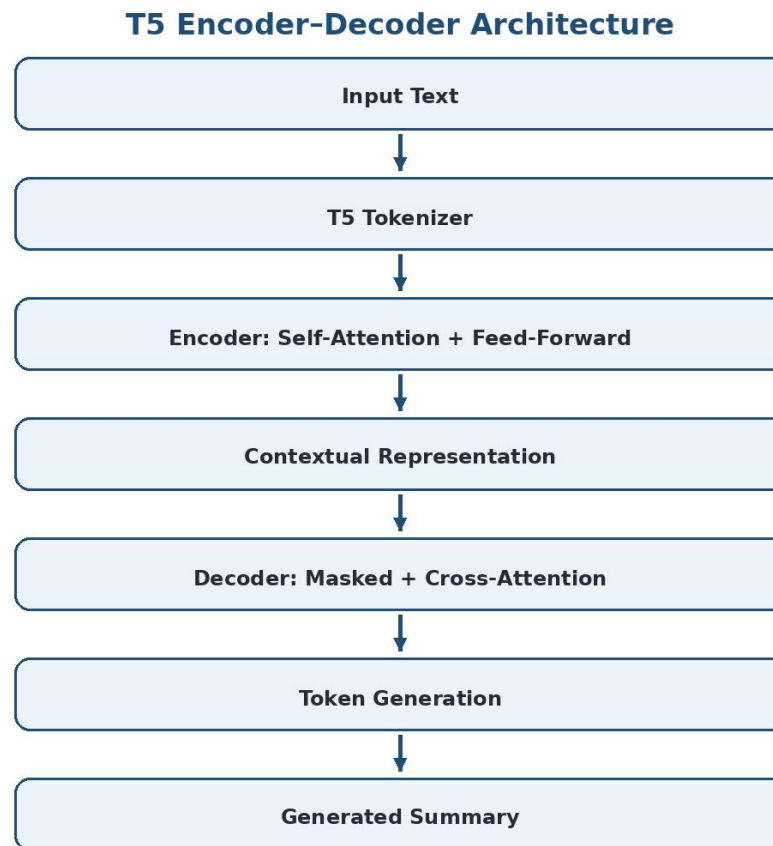


**Figure 2.2: Transformer-Based Text Summarization Architecture**

Transformers use attention mechanisms to model relationships among tokens. The project material distinguishes BERT, GPT and T5 by their architecture and intended role. BERT is primarily an encoder model focused on representation and understanding. GPT is decoder-oriented and designed for generation. T5 uses an encoder-decoder structure, making it naturally suitable for conditional generation tasks such as summarization.

The use of pretrained models is an engineering advantage because transfer learning reduces the amount of task-specific training required compared with building and training a large transformer from scratch. The trade-off is that inference and fine-tuning still require substantial computational resources depending on model size and workload.

## 2.5 T5 and Multilingual T5



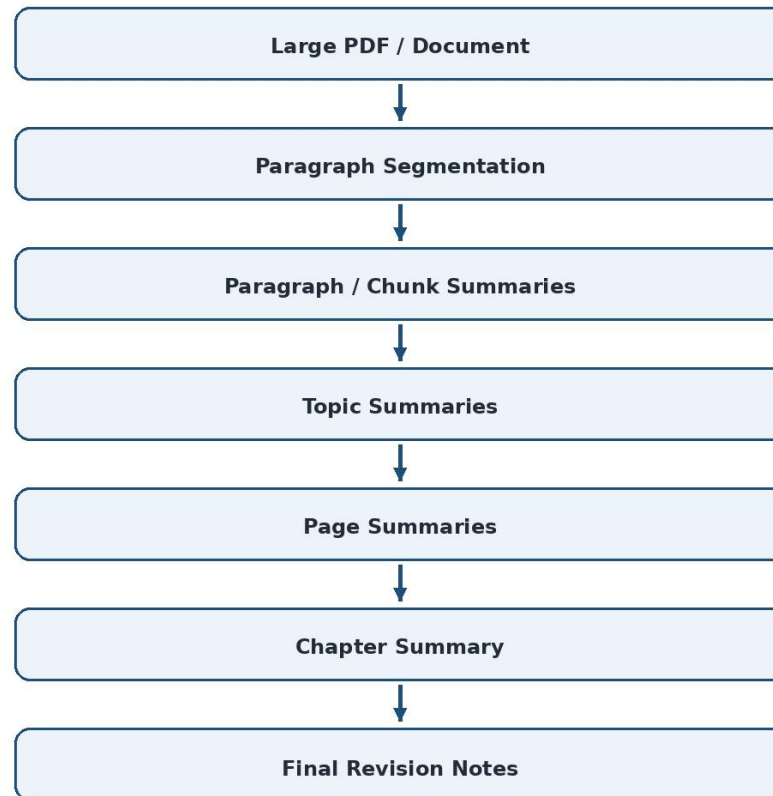
**Figure 2.4: T5 Transformer Encoder–Decoder Architecture**

T5 formulates NLP tasks in a text-to-text manner. For summarization, the input is converted into an instruction-style or task-specific sequence and the decoder generates the target summary. A multilingual T5-family model can extend this concept across languages using a shared multilingual representation and tokenizer.

The project uses the MLSUM/MLSAM dataset as the primary multilingual summarization dataset. The supplied project material states that it contains more than 1.5 million article–summary pairs across French, German, Spanish, Russian and Turkish and is organized into training, validation and test splits.

## 2.6 Long-Document Summarization

### Hierarchical Summarization for Long Documents

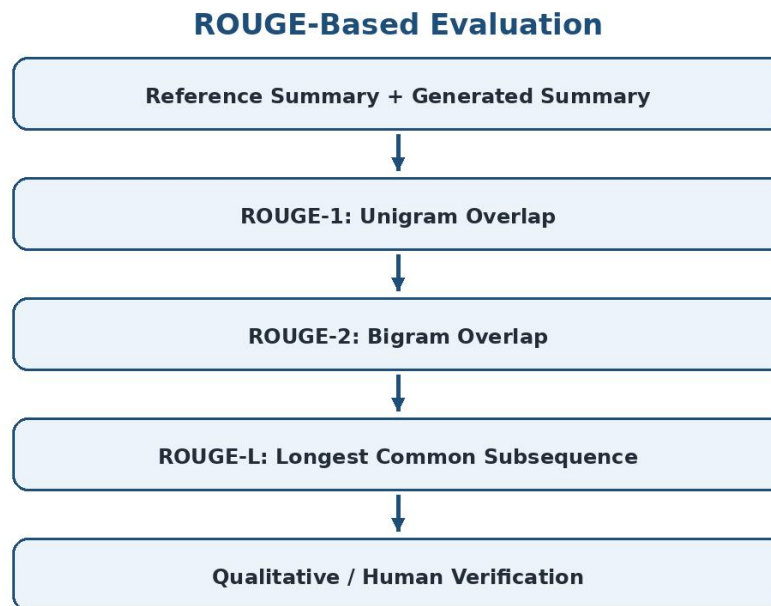


**Figure 2.5: Hierarchical Summarization for Long Documents**

A major engineering issue is input length. A full 80-page PDF cannot simply be placed into one model input without exceeding practical context limits. The proposed system therefore uses hierarchical summarization. It first summarizes smaller paragraphs, then combines those micro-summaries into topic notes, page notes and chapter-level notes.

This approach introduces additional model calls and possible error accumulation, but it creates a controllable hierarchy and allows the application to present different levels of detail. It also makes source mapping more manageable because each intermediate representation can retain its source relationship.

## 2.7 Evaluation with ROUGE

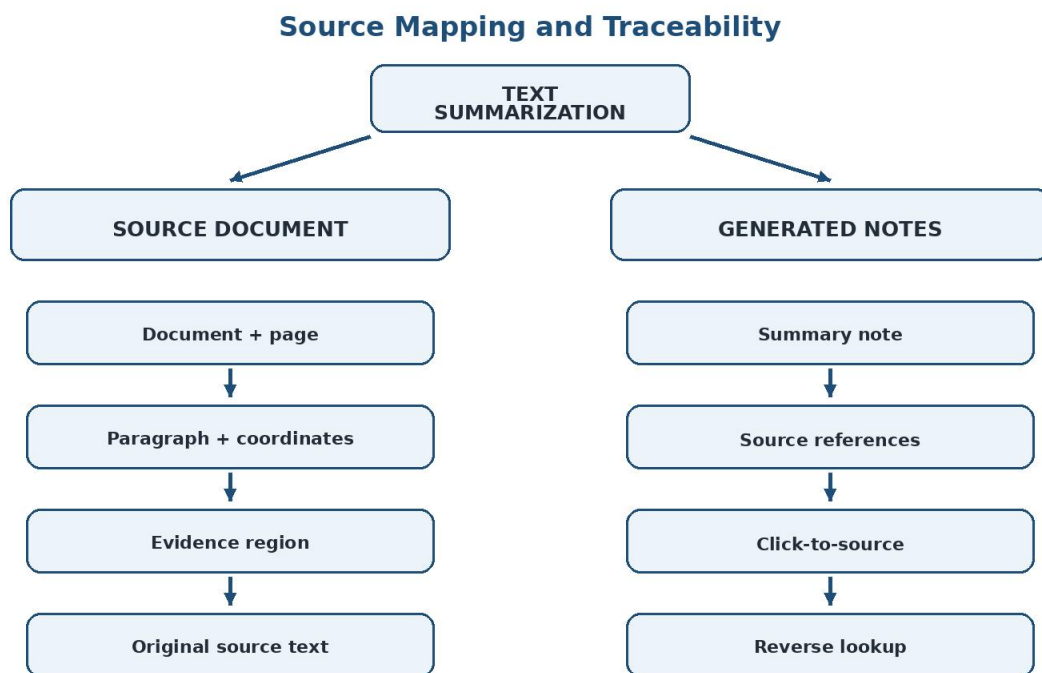


**Figure 2.8: ROUGE-Based Summary Evaluation Process**

ROUGE is used to compare generated summaries with human-written reference summaries. ROUGE-1 measures overlap of individual tokens or unigrams, ROUGE-2 measures bigram overlap, and ROUGE-L uses the longest common subsequence to capture a broader sequence relationship.

ROUGE is useful because it provides a repeatable numerical evaluation, but it does not completely represent factual correctness, readability or semantic equivalence. The project material explicitly notes that human evaluation remains important because different metrics can produce different conclusions and word overlap can miss meaning.

## 2.8 Traceable / Source-Grounded Learning Systems



**Figure 2.7: Source Mapping Between Generated Summary and Original Document**

The proposed platform extends summarization into source-grounded learning. Instead of storing only generated text, it stores page numbers, paragraph identifiers and text-position coordinates. When a learner selects a note, the application can locate the corresponding source region and highlight it in the PDF viewer.

This changes the role of summarization from a final answer generator into a navigable representation of a document. It also provides a mechanism for students to verify definitions, formulas, conditions and exceptions before relying on a generated note.

## 2.9 Comparative Analysis

| Aspect               | Traditional Extractive       | Generic Abstractive | Proposed T5 Platform         |
|----------------------|------------------------------|---------------------|------------------------------|
| Output               | Selected sentences<br>source | Generated text      | Generated hierarchical notes |
| Rewriting            | No                           | Yes                 | Yes                          |
| Multilingual support | Depends on method            | Model dependent     | MLSUM/MLSAM + T5-family      |

|                     |                                |                           |                                                  |
|---------------------|--------------------------------|---------------------------|--------------------------------------------------|
| Context modeling    | Limited                        | Strong                    | Transformer-based                                |
| Long documents      | Possible with ranking          | Context limited           | Hierarchical processing                          |
| Source traceability | Naturally available            | Often weak                | Explicit<br>page/paragraph/coordinate<br>mapping |
| Personalization     | Usually absent                 | Usually limited           | Planned<br>confusion/learning<br>analytics       |
| Evaluation          | Precision/coverage<br>variants | ROUGE and human<br>review | ROUGE + source<br>verification                   |

## 2.10 Engineering Gap

The engineering gap identified by the project is the separation between summary generation and trustworthy academic use. A generic summarizer can produce fluent output, but an academic learning platform needs additional structure: document segmentation, source mapping, multiple levels of summary, reusable notes and mechanisms for checking the generated information. The project therefore combines the NLP model with an evidence-oriented document representation.

## CHAPTER 3

### ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

#### 3.1 System Requirements

##### 3.1.1 Functional Requirements

| ID    | Functional Requirement | Description                                                         | Priority |
|-------|------------------------|---------------------------------------------------------------------|----------|
| FR-01 | Document Input         | Accept supported PDF/text learning material.                        | High     |
| FR-02 | PDF Extraction         | Extract text, page structure, paragraph boundaries and coordinates. | High     |
| FR-03 | Preprocessing          | Clean noise, normalize text and segment content.                    | High     |
| FR-04 | Topic Segmentation     | Detect headings and group related content.                          | High     |
| FR-05 | T5 Summarization       | Generate abstractive summaries from processed chunks.               | High     |
| FR-06 | Hierarchy              | Create paragraph, topic, page and chapter summaries.                | High     |
| FR-07 | Source Mapping         | Associate notes with source pages, paragraphs and coordinates.      | High     |
| FR-08 | Evaluation             | Calculate ROUGE-1, ROUGE-2 and ROUGE-L.                             | High     |
| FR-09 | Search                 | Support keyword and semantic retrieval across source and notes.     | Medium   |
| FR-10 | Study Workspace        | Display PDF and notes in a linked interface.                        | High     |

|       |                    |                                                        |        |
|-------|--------------------|--------------------------------------------------------|--------|
| FR-11 | Editing            | Allow notes to be edited, annotated and pinned.        | Medium |
| FR-12 | Learning Analytics | Collect interaction signals for confusion analysis.    | Medium |
| FR-13 | Assistant          | Provide retrieval-grounded clarification/viva support. | Medium |

### 3.1.2 Non-Functional Requirements

| Quality Attribute | Requirement                | Engineering Interpretation                                           |
|-------------------|----------------------------|----------------------------------------------------------------------|
| Performance       | Reasonable processing time | Use chunking, batching and background jobs for large documents.      |
| Usability         | Simple study workflow      | Keep PDF and notes visible together.                                 |
| Reliability       | Consistent source mapping  | Never discard location metadata during preprocessing.                |
| Maintainability   | Modular implementation     | Separate extraction, preprocessing, model, evaluation and UI layers. |
| Scalability       | Support larger documents   | Use hierarchical processing and asynchronous jobs.                   |
| Security          | Protect uploaded content   | Authentication, authorization and secure secret handling.            |
| Explainability    | Allow verification         | Provide exact source links for generated notes.                      |
| Portability       | Deployment flexibility     | Containerization and environment-variable configuration.             |

### 3.2 Constraints and Applicable Standards

The project is a software/NLP capstone and therefore has constraints associated with model context length, available computing resources, dataset quality, inference time, data privacy and the reliability of generated text. These constraints are addressed as design requirements rather than ignored during implementation.

| Reference / Principle       | Relevance to Project             | Application                                                                                |
|-----------------------------|----------------------------------|--------------------------------------------------------------------------------------------|
| ISO/IEC 25010               | Software quality characteristics | Consider functionality, performance, usability, reliability, security and maintainability. |
| IEEE 29148 principles       | Requirements engineering         | Document stakeholder needs, functional requirements and acceptance criteria.               |
| Responsible AI principles   | Generated-content risk           | Make outputs verifiable and avoid presenting generated text as unquestionable fact.        |
| Data minimization principle | Uploaded documents               | Store only information required for the learning workflow.                                 |
| Secure software practice    | Authentication and secrets       | Protect JWT secrets, database credentials and external API keys.                           |

### 3.3 Design Specifications

| Component  | Selected Technology       | Design Reason                                                  |
|------------|---------------------------|----------------------------------------------------------------|
| Frontend   | Vite + React + TypeScript | Component-based interactive interface.                         |
| Styling    | TailwindCSS               | Rapid construction of notebook and split-screen layouts.       |
| PDF Viewer | react-pdf / pdf.js        | Render source documents and support coordinate-based overlays. |

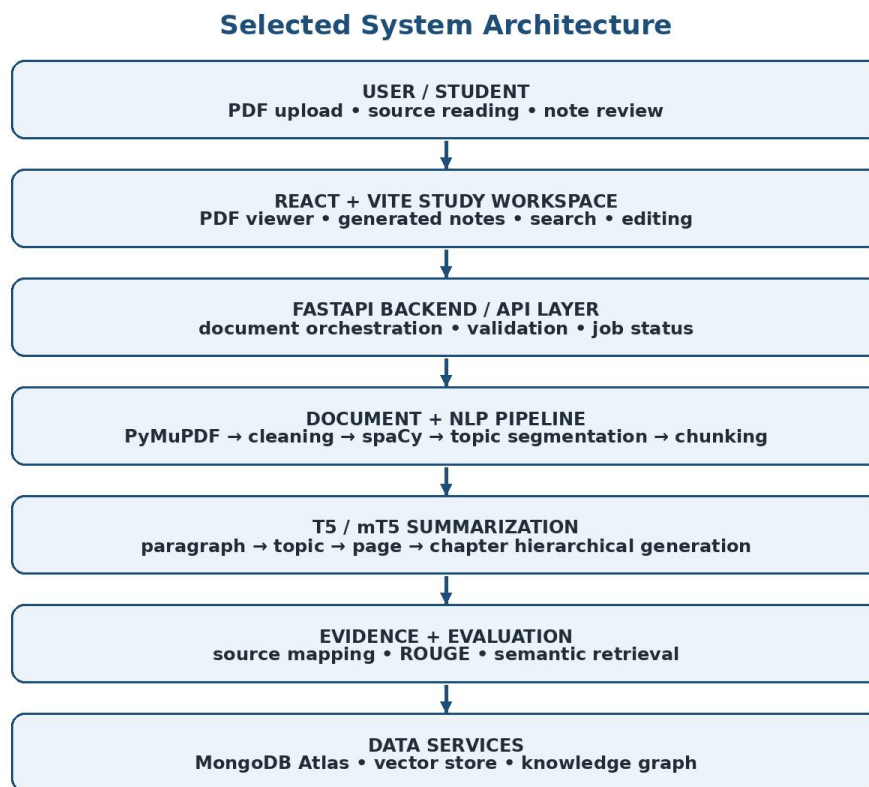
|                 |                                   |                                                                         |
|-----------------|-----------------------------------|-------------------------------------------------------------------------|
| Panel Layout    | react-resizable-panels            | Resizable source/notes workspace.                                       |
| Graph UI        | react-flow                        | Interactive concept and relationship visualization.                     |
| State           | Zustand                           | Shared state such as selected note and active highlight.                |
| Backend         | FastAPI + Pydantic                | Python-native API around NLP/ML components.                             |
| Background Jobs | Celery + Redis                    | Long-running document processing.                                       |
| Database        | MongoDB Atlas                     | Flexible nested document/note metadata.                                 |
| PDF Extraction  | PyMuPDF                           | Text plus exact bounding-box coordinates.                               |
| Preprocessing   | spaCy                             | Sentence splitting, cleaning and pattern detection.                     |
| Summarization   | T5/mT5 via Transformers + PyTorch | Abstractive encoder-decoder generation.                                 |
| Embeddings      | sentence-transformers             | Semantic retrieval and confidence-related features.                     |
| Vector Search   | ChromaDB / FAISS                  | Fast retrieval of relevant chunks.                                      |
| Knowledge Graph | NetworkX                          | Concept and relationship construction.                                  |
| Assistant       | Groq API                          | Conversational clarification only; core summarization remains T5-based. |

### 3.4 Alternative Solutions and Evaluation

| Alternative           | Advantages                            | Disadvantages                   | Decision      |
|-----------------------|---------------------------------------|---------------------------------|---------------|
| Extractive summarizer | Simple, fast, naturally source-linked | Cannot paraphrase or synthesize | Baseline only |

|                         |                                                 |                                                      |                                        |
|-------------------------|-------------------------------------------------|------------------------------------------------------|----------------------------------------|
| Single-pass document T5 | Simple pipeline                                 | Sequence-length limitation; high memory/input burden | Rejected                               |
| Large T5 model          | Potentially stronger generation                 | High compute and inference cost                      | Not preferred for realistic deployment |
| T5-small/base family    | Transfer learning with lower cost               | May require careful tuning                           | Preferred                              |
| Generic PDF chatbot     | Easy question-answer interaction                | Does not build persistent hierarchical notes         | Not sufficient as core system          |
| Hierarchical T5         | Long-document handling + multiple granularities | More processing stages                               | Selected                               |
| Relational database     | Strong tabular consistency                      | Less natural for nested note/graph structures        | Not primary choice                     |
| MongoDB                 | Flexible nested document representation         | Requires careful schema discipline                   | Selected                               |

### 3.5 Selected Architecture



**Figure 3.1: Selected System Architecture**

The selected architecture is layered so that each major engineering concern can be developed and tested independently. The document layer extracts source content and coordinates. The NLP layer cleans and segments the text. The summarization layer generates abstractive content. The evidence layer preserves the relationship between generated notes and source regions. The application layer presents the results and supports interactive learning.

High-level flow:

**PDF / Text Input → Extraction → Preprocessing → Topic Segmentation → Chunking → T5 Summarization → Hierarchical Roll-up → Source Mapping → Evaluation → Interactive Learning Workspace**

### 3.5.1 Frontend Layer

The frontend is planned around React and Vite using TypeScript. The PDF is rendered using react-pdf/pdf.js. Generated notes are presented in a notebook-like interface. A resizable panel separates the source PDF from the generated notes, allowing users to change the amount of screen space given to each view.

### 3.5.2 Backend Layer

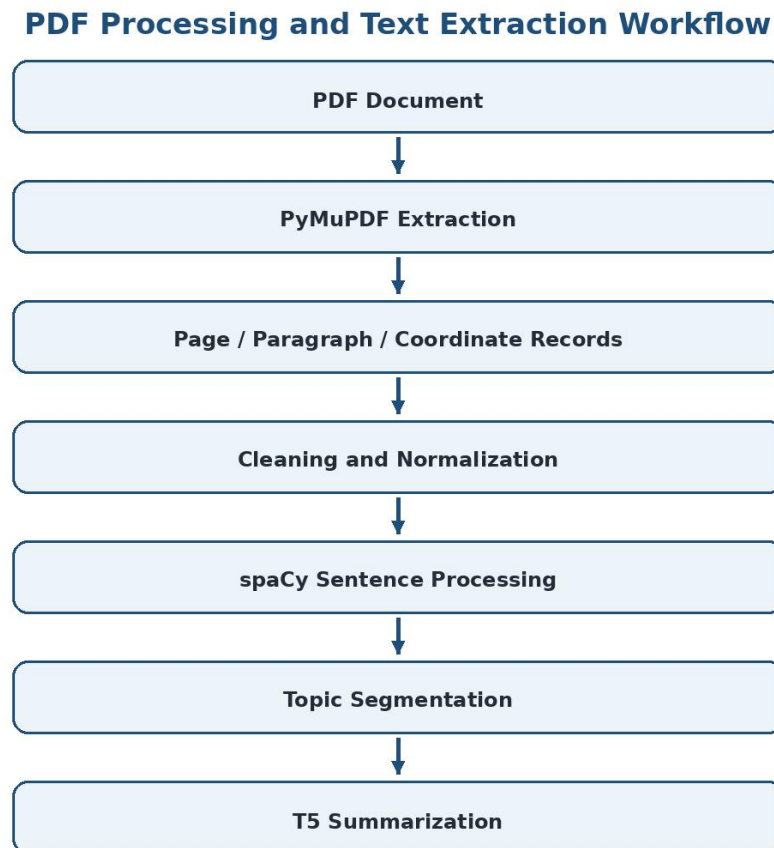
FastAPI provides Python-native API endpoints around document processing and model inference. Pydantic is used to validate structured information such as page numbers, paragraph identifiers and coordinates. Long-running jobs can be moved to Celery workers with Redis acting as the message broker so that a large PDF does not block a normal web request.

### 3.5.3 Data Layer

MongoDB Atlas is selected for flexible storage of document metadata, extracted chunks, hierarchical notes and graph-related information. The project's data structures are naturally nested: a chapter contains pages, pages contain topics and topics contain paragraphs, while a note may reference one or more source regions.

## 3.6 Detailed NLP Pipeline Design

### 3.6.1 Stage 1 – PDF Extraction



**Figure 3.2: PDF Document Processing and Text Extraction Workflow**

PyMuPDF reads the source PDF and extracts text along with page-level and text-span information. The critical engineering requirement is to retain bounding-box coordinates because later highlighting depends on knowing where a source span appears on the rendered page.

### 3.6.2 Stage 2 – Cleaning and Normalization

Extracted PDF text may contain line breaks inside sentences, repeated headers and footers, page numbers, inconsistent spaces and extraction artifacts. The preprocessing stage removes or normalizes these patterns while preserving the mapping to the original page and paragraph. The system therefore separates 'clean text for the model' from 'source metadata for evidence'.

### 3.6.3 Stage 3 – Sentence Processing

spaCy is used for sentence splitting and rule-based processing. Sentence boundaries are required because chunking at arbitrary character positions can split logical statements. The preprocessing stage also provides a foundation for detecting definitions, formulas, units, laws, examples and other exam-relevant patterns.

### 3.6.4 Stage 4 – Topic Segmentation

Headings and subheadings are used to organize related paragraphs into topics and subtopics. Topic segmentation is important because the later hierarchy should represent the document's logical structure rather than simply its raw page order.

### 3.6.5 Stage 5 – T5 Generation

The processed text is tokenized using the model's tokenizer and supplied to the T5-family encoder-decoder. The encoder builds contextual representations of the input, while the decoder generates the summary token by token. The project uses abstractive generation so that the output can condense and rewrite information.

### 3.6.6 Stage 6 – Hierarchical Roll-up

Paragraph summaries are aggregated into topic notes. Topic notes are then used to construct page-level summaries, and page-level information is rolled into chapter-level revision notes. This bottom-up process is deliberately chosen because it respects the practical limitations of transformer input size and creates multiple useful study views.

## 3.7 Source-Mapping and Traceability Design

Traceability is the main differentiating design decision. Every chunk entering the NLP pipeline carries metadata that identifies its document, page, paragraph and text coordinates. The generated note stores references to those source records. The application can therefore translate a selected note into one or more highlight regions in the original PDF.

| Metadata Field | Purpose                     | Example Meaning        |
|----------------|-----------------------------|------------------------|
| document_id    | Identifies source document  | Uploaded textbook/unit |
| page_number    | Identifies source page      | Page 27                |
| paragraph_id   | Identifies source paragraph | Paragraph 27-04        |
| bbox           | Identifies visual position  | x0, y0, x1, y1         |

|               |                             |                              |
|---------------|-----------------------------|------------------------------|
| topic_id      | Connects paragraph to topic | Topic 5                      |
| note_id       | Identifies generated note   | Note 5.3                     |
| source_refs   | Maps note to evidence       | List of paragraph IDs        |
| summary_level | Identifies hierarchy        | paragraph/topic/page/chapter |

When a user clicks a generated note, the frontend uses the stored source reference to locate the page and draw a highlight overlay over the stored coordinates. Reverse lookup follows the opposite direction: a selected paragraph can identify the notes that summarize it.

### 3.8 Data Model and Information Flow

The information flow is intentionally designed so that source information is never detached from processed content. Raw extraction produces source records. Preprocessing produces cleaned chunks while retaining source references. Summarization produces generated text with source references. Hierarchical aggregation combines notes while retaining links to the underlying evidence.

| Object          | Important Fields                        | Relationship                  |
|-----------------|-----------------------------------------|-------------------------------|
| Document        | id, title, page_count, metadata         | Contains pages/chunks         |
| Source Chunk    | page, paragraph, text, bbox, topic_id   | Evidence unit                 |
| Summary Note    | id, text, level, source_refs            | Summarizes one or more chunks |
| Topic           | id, heading, parent, note_refs          | Groups related chunks/notes   |
| Graph Node      | id, label, type, evidence_refs          | Represents concept            |
| Graph Edge      | source, target, relation, evidence_refs | Represents relationship       |
| Learning Signal | student, source_id, event, timestamp    | Feeds personalization         |

### 3.9 Application and Interface Design

The proposed interface uses a split-screen layout. The left side displays the original PDF and evidence highlights. The right side displays generated notes organized by topic. A draggable divider allows the user to prioritize either source reading or note review.

- Click-to-source: selecting a note highlights its supporting PDF paragraph.

- Reverse lookup: selecting a source paragraph reveals its related note.
- Multi-granularity views: paragraph, topic, page and chapter.
- Editable notes: learners can modify generated text and add annotations.
- Personal notebook: pinned content can be collected for revision.
- Universal search: search source content and generated notes.
- Style switching: concise revision points, detailed notes or simplified explanations.
- Export: notes can be prepared for offline PDF or Markdown use.

### 3.10 Trade-off Analysis

| Design Decision             | Benefit                                     | Cost / Risk                        | Reason for Selection                    |
|-----------------------------|---------------------------------------------|------------------------------------|-----------------------------------------|
| T5 over extractive baseline | Abstractive, coherent rewriting             | Higher compute and generation risk | Core project requirement                |
| Hierarchical summarization  | Supports long documents and multiple levels | More inference stages              | Avoids single-input length problem      |
| Source coordinates          | Pixel-level evidence highlighting           | More metadata and storage          | Critical for trust                      |
| MongoDB                     | Flexible nested structures                  | Schema discipline required         | Fits notes/graphs                       |
| FastAPI                     | Python-native ML integration                | Requires async job strategy        | Reduces language boundary               |
| Celery + Redis              | Non-blocking long jobs                      | Operational complexity             | Needed for larger documents             |
| Separate embedding model    | Better semantic retrieval                   | Additional model/storage           | T5 is not primarily an embedding model  |
| Groq assistant boundary     | Interactive conversation                    | External dependency                | Keeps core NLP contribution independent |

### 3.11 Sustainability

Sustainability is addressed primarily through computational efficiency and software lifecycle design. Transformer training and inference can consume substantial compute. The use of pretrained

models avoids the need to train a large model from scratch. The architecture also allows smaller T5-family checkpoints to be used for practical development and deployment.

- Use pretrained models and transfer learning.
- Use batching and chunk-level processing where appropriate.
- Use smaller model variants when their quality is adequate.
- Introduce background processing only when necessary for long jobs.
- Consider quantization, pruning or distillation in future optimization.
- Keep deployment configuration reproducible through Docker and environment variables.

### 3.12 Ethics and Responsible AI

The central ethical issue is that generated text may appear authoritative even when it is incomplete or incorrect. The project therefore treats verification as a system feature. Source links should be visible when a generated note is used for academic study.

- Do not present generated summaries as guaranteed factual truth.
- Preserve source evidence so learners can verify important claims.
- Clearly separate the core T5 summarizer from the optional conversational assistant.
- Do not use student interaction data beyond what is necessary for personalization.
- Protect uploaded academic documents and avoid unnecessary exposure of private information.
- Retain human review as part of high-stakes academic interpretation.

### 3.13 Safety and Reliability

For a software-only educational system, safety is primarily concerned with information reliability, predictable operation and avoidance of misleading output. A fluent summary that drops a critical condition can be more dangerous academically than a visibly poor summary. The system should therefore encourage users to inspect source evidence, especially for formulas, rules, exceptions and definitions.

### 3.14 Security and Privacy

| Security Area  | Threat                             | Mitigation                     |
|----------------|------------------------------------|--------------------------------|
| Authentication | Unauthorized account access        | JWT-based authentication       |
| Authorization  | Access to another user's documents | User/document ownership checks |
| Secrets        | API/database key exposure          | Environment variables and      |

|               |                                   |                                                               |
|---------------|-----------------------------------|---------------------------------------------------------------|
|               |                                   | secure deployment secrets                                     |
| Database      | Unauthorized data access          | Protected MongoDB configuration and least privilege           |
| Uploaded PDFs | Sensitive content leakage         | Controlled storage and access                                 |
| API           | Malicious or invalid requests     | Pydantic validation and input checks                          |
| External AI   | Unintended source-data disclosure | Retrieve only necessary context and maintain project boundary |

### 3.15 Risk Register and Mitigation

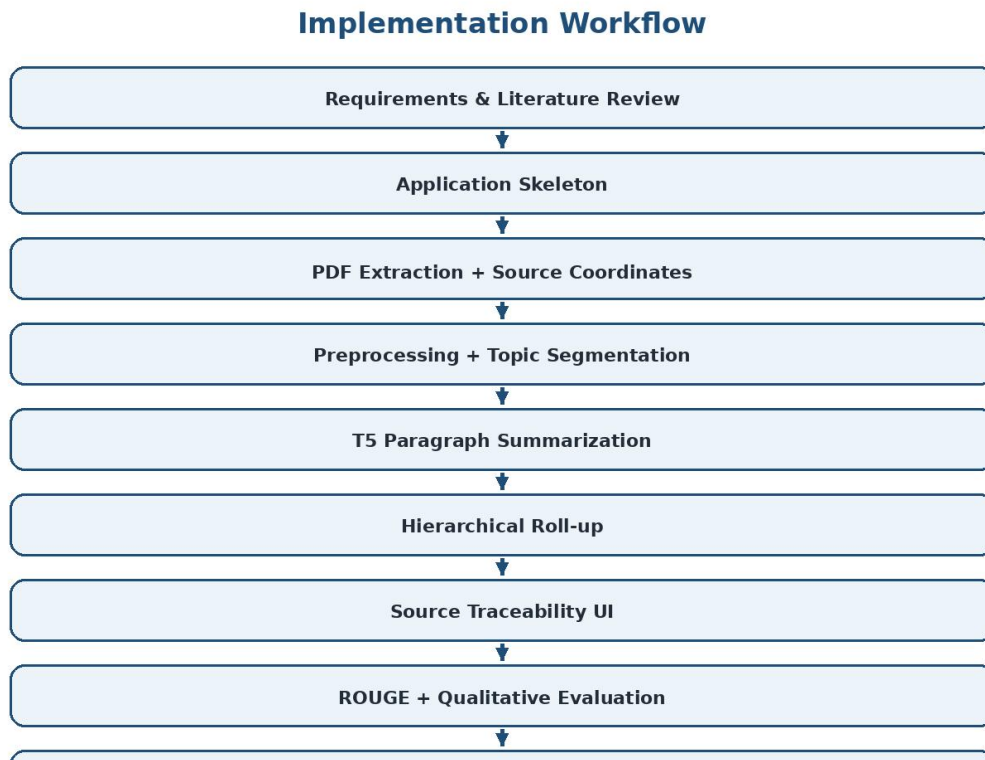
| Risk                               | Likelihood | Impact     | Mitigation                                                | Residual Risk |
|------------------------------------|------------|------------|-----------------------------------------------------------|---------------|
| Model input exceeds context limit  | High       | High       | Chunking and hierarchical summarization                   | Medium        |
| Important information omitted      | Medium     | High       | Source mapping + evaluation + human verification          | Medium        |
| Hallucinated statement             | Medium     | High       | Evidence links and retrieval-grounded assistance          | Medium        |
| Poor language-specific performance | Medium     | Medium     | Language-wise testing and dataset analysis                | Medium        |
| Slow inference                     | Medium     | Medium     | Smaller models, batching and async jobs                   | Low/Medium    |
| PDF extraction error               | Medium     | High       | Extraction validation and source-preserving preprocessing | Medium        |
| Unauthorized document access       | Low        | High       | JWT, authorization and protected storage                  | Low/Medium    |
| External API unavailable           | Medium     | Low/Medium | Keep core summarizer independent of assistant layer       | Low           |
| Evaluation metric misleading       | Medium     | Medium     | Use ROUGE with                                            | Medium        |

|  |  |  |                   |  |
|--|--|--|-------------------|--|
|  |  |  | qualitative/human |  |
|--|--|--|-------------------|--|

# CHAPTER 4

## IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

### 4.1 Development Approach



**Figure 4.1: End-to-End Implementation Workflow**

The project follows an incremental development strategy. The application skeleton and document pipeline are established before adding advanced AI features. The recommended implementation sequence begins with the frontend, backend, database connection and authentication, followed by PDF extraction and source highlighting. Preprocessing and topic segmentation are then validated before T5 summarization is introduced.

10. Set up Vite + React frontend, FastAPI backend and MongoDB Atlas connection.
11. Implement basic authentication and document upload.
12. Build PDF extraction using PyMuPDF and store source coordinates.
13. Verify coordinate-based highlighting before adding machine learning.
14. Implement spaCy-based preprocessing and topic segmentation.
15. Implement paragraph-level T5 summarization.

16. Store generated summaries together with source references.
17. Build topic → page → chapter hierarchical roll-up.
18. Connect notes to PDF highlights and reverse lookup.
19. Add key-information extraction and exam-essentials notebook.
20. Add semantic search and retrieval-grounded assistant.
21. Add confusion analytics, knowledge graph and viva simulator.
22. Introduce Celery/Redis for large background jobs.
23. Containerize and deploy only after the local workflow is stable.

## 4.2 Implementation Environment

| Layer              | Technology / Tool                      | Role                                      |
|--------------------|----------------------------------------|-------------------------------------------|
| Programming        | Python                                 | NLP and backend implementation            |
| Deep Learning      | PyTorch                                | Model computation                         |
| Transformer        | Hugging Face Transformers              | T5/mT5 model and tokenizer                |
| Data               | Pandas, NumPy                          | Dataset processing                        |
| PDF                | PyMuPDF                                | Text and coordinate extraction            |
| NLP                | spaCy                                  | Sentence processing and pattern detection |
| Evaluation         | ROUGE                                  | Summary quality measurement               |
| Frontend           | React + Vite + TypeScript              | User interface                            |
| Backend            | FastAPI + Pydantic                     | API and orchestration                     |
| Async              | Celery + Redis                         | Background processing                     |
| Database           | MongoDB Atlas                          | Metadata and notes                        |
| Semantic Retrieval | sentence-transformers + ChromaDB/FAISS | Retrieval                                 |
| Graph              | NetworkX + react-flow                  | Knowledge graph                           |
| Assistant          | Groq API                               | Grounded conversation                     |

### 4.3 Dataset Preparation

The project uses the MLSUM/MLSAM multilingual summarization dataset. The supplied project material describes it as a large dataset of multilingual news article–summary pairs, with more than 1.5 million pairs across French, German, Spanish, Russian and Turkish. Human-written summaries provide reference targets for supervised abstractive summarization.

The dataset preparation process consists of loading records, checking required article and summary fields, removing invalid or empty records, normalizing text where appropriate, tokenizing inputs and targets, and maintaining train/validation/test separation. Because multilingual text may contain different scripts and word structures, subword tokenization is important.

| Preparation Step  | Purpose                       | Expected Output                     |
|-------------------|-------------------------------|-------------------------------------|
| Record validation | Remove malformed examples     | Valid article-summary pairs         |
| Text cleaning     | Reduce irrelevant noise       | Normalized source/target            |
| Language handling | Preserve multilingual content | Language-aware records              |
| Tokenization      | Convert text to model tokens  | Input/target token IDs              |
| Length filtering  | Respect model limits          | Usable training/evaluation examples |
| Dataset split     | Avoid leakage between stages  | Train/validation/test sets          |

### 4.4 PDF Extraction and Preprocessing

For the application workflow, PyMuPDF extracts readable text and coordinates from uploaded PDFs. The extraction process must preserve enough structure to distinguish headings, paragraphs and pages. The system keeps source metadata even when the visible text is cleaned for NLP processing.

The preprocessing stage handles common PDF extraction artifacts such as broken lines, repeated headers, page numbers and inconsistent whitespace. spaCy provides sentence splitting, while rule-based patterns can support identification of definitions, formulas, units and other exam-oriented information.

## 4.5 T5 Summarization Implementation

The T5-family model is loaded through Hugging Face Transformers. A tokenizer converts source text into model-compatible token sequences. The encoder processes the input and builds contextual representations. The decoder then generates the summary sequentially.

The implementation can use a pretrained checkpoint for initial development and a fine-tuned checkpoint for the project's supervised summarization task. Model size is a practical engineering decision: larger models may provide stronger capacity but increase inference time and resource requirements. The project architecture therefore favors realistic model sizes for development and deployment.

### 4.5.1 Generation Process

24. Receive a cleaned text chunk.
25. Apply the T5 tokenizer.
26. Create model input tensors and attention masks.
27. Pass the input through the encoder-decoder model.
28. Generate output tokens using configured decoding parameters.
29. Decode tokens back into readable text.
30. Store the generated summary with source metadata.

## 4.6 Hierarchical Summarization Implementation

The hierarchical summarizer is the core long-document strategy. A large document is first divided into logical paragraphs or manageable chunks. Each chunk receives a micro-summary. Micro-summaries belonging to a topic are then combined and summarized again. The same process is repeated for page and chapter levels.

| Level     | Input                      | Output          | Purpose                    |
|-----------|----------------------------|-----------------|----------------------------|
| Paragraph | Original paragraph         | Micro-summary   | Fine-grained compression   |
| Topic     | Paragraph summaries        | Topic note      | Concept-level organization |
| Page      | Topic notes / page content | Page note       | Page-level revision        |
| Chapter   | Page notes                 | Chapter summary | High-level revision        |

The hierarchy provides an important usability benefit: a student can begin with a chapter-level overview and progressively inspect a topic or paragraph when more detail is required. The same hierarchy also provides natural units for source mapping.

#### 4.7 Source Grounding Implementation

Each summary record contains a source reference list. For paragraph-level summaries, this may point directly to one source paragraph. Higher-level summaries can point to multiple lower-level notes and therefore indirectly to multiple source regions. The source-mapping engine resolves these relationships to page and coordinate information for visual highlighting.

This design supports two directions of navigation. Forward navigation begins with a note and opens its supporting source. Reverse navigation begins with a source paragraph and opens the generated note associated with it.

#### 4.8 Application Integration

The React interface receives document and note metadata through the FastAPI backend. The PDF viewer renders the source document. When a note becomes active, the frontend requests or uses the stored page and coordinate information to render a highlight overlay. The notes panel displays the corresponding generated text.

For large documents, the backend can delegate processing to Celery workers. Redis acts as the message broker. The user interface can then show stages such as extraction, segmentation and summarization instead of appearing unresponsive during a long-running job.

#### 4.9 Testing Strategy

Testing is organized at unit, integration and end-to-end levels. Unit testing verifies individual functions such as cleaning, sentence segmentation and metadata handling. Integration testing verifies that extracted source records can be passed through preprocessing and summarization while retaining source references. End-to-end testing verifies the full workflow from upload to displayed notes and evidence highlighting.

#### 4.10 Functional Test Cases

| TC ID | Test Scenario | Input        | Expected Result | Status         |
|-------|---------------|--------------|-----------------|----------------|
| TC-01 | Upload valid  | Readable PDF | Document        | To be recorded |

|       |                            |                               |                                           |                |
|-------|----------------------------|-------------------------------|-------------------------------------------|----------------|
|       | PDF                        |                               | accepted and extracted                    |                |
| TC-02 | Extract page text          | PDF page                      | Text associated with correct page         | To be recorded |
| TC-03 | Preserve coordinates       | Text span                     | Bounding box stored                       | To be recorded |
| TC-04 | Clean extraction noise     | PDF with headers/line breaks  | Normalized text                           | To be recorded |
| TC-05 | Segment topic              | Headings + paragraphs         | Logical topic groups                      | To be recorded |
| TC-06 | Generate paragraph summary | Clean paragraph               | Abstractive summary returned              | To be recorded |
| TC-07 | Generate hierarchy         | Paragraph summaries           | Topic/page/chapter notes                  | To be recorded |
| TC-08 | Click-to-source            | Generated note                | Correct source highlighted                | To be recorded |
| TC-09 | Reverse lookup             | Source paragraph              | Related note displayed                    | To be recorded |
| TC-10 | ROUGE evaluation           | Generated + reference summary | ROUGE-1/2/L values                        | To be recorded |
| TC-11 | Semantic search            | Keyword/concept query         | Relevant source chunks                    | To be recorded |
| TC-12 | Invalid input handling     | Unsupported/empty file        | Validation error                          | To be recorded |
| TC-13 | Long document              | Large PDF                     | Background processing without UI blocking | To be recorded |
| TC-14 | Authentication             | Valid/invalid credentials     | Authorized access only                    | To be recorded |

## 4.11 Evaluation Method

The principal automatic evaluation method is ROUGE. Generated summaries are compared against human-written reference summaries. ROUGE-1 provides unigram overlap, ROUGE-2 provides bigram overlap and ROUGE-L measures longest-common-subsequence similarity.

The project deliberately avoids treating ROUGE as a complete measure of quality. A generated summary may have low word overlap while preserving meaning through paraphrasing, or it may have high overlap while still missing an important condition. Therefore, qualitative review and source verification should accompany metric-based evaluation.

## 4.12 Results and Analysis

The supplied project material confirms the use of T5-based abstractive summarization, MLSUM/MLSAM multilingual data and ROUGE-1/2/L evaluation. It also defines the source-traceability and hierarchical-note architecture. However, the supplied material does not provide a verified final numerical ROUGE table, latency benchmark or completed test-count record. Accordingly, this report does not fabricate those values.

| Measure                | Evidence Available from Project Material                      | Final Submission Action                 |
|------------------------|---------------------------------------------------------------|-----------------------------------------|
| Summary generation     | T5-family abstractive generation is part of the system design | Attach representative generated outputs |
| ROUGE-1                | Defined as an evaluation metric                               | Insert actual measured score            |
| ROUGE-2                | Defined as an evaluation metric                               | Insert actual measured score            |
| ROUGE-L                | Defined as an evaluation metric                               | Insert actual measured score            |
| Long-document handling | Hierarchical paragraph → topic → page → chapter design        | Attach end-to-end demonstration         |
| Source traceability    | Page/paragraph/coordinate mapping design                      | Attach click-to-source screenshot       |
| Processing time        | No verified benchmark                                         | Measure on final                        |

|                |                        |                                     |
|----------------|------------------------|-------------------------------------|
|                | supplied               | hardware/software setup             |
| Test pass rate | Test scenarios defined | Insert actual executed test results |

#### 4.13 Comparison with Existing Approaches

| Aspect             | Traditional / Generic Method             | Proposed System                                         |
|--------------------|------------------------------------------|---------------------------------------------------------|
| Summarization      | Extractive or free-form generated text   | T5 abstractive summarization                            |
| Long documents     | May truncate or require manual splitting | Hierarchical processing                                 |
| Source evidence    | Often limited                            | Explicit source mapping                                 |
| Study structure    | Single summary                           | Paragraph/topic/page/chapter                            |
| Personalization    | Usually absent                           | Confusion and interaction analytics planned             |
| Retrieval          | Question-focused                         | Persistent document representation + semantic retrieval |
| Learning artifacts | Usually not available                    | Notebook, exam essentials, graph and transition cards   |
| Assistant          | General chatbot                          | Retrieval-grounded assistant over source content        |

#### 4.14 Strengths and Limitations

##### Strengths

- Transformer-based abstractive generation.
- Multilingual dataset and model orientation.
- Hierarchical summarization for long documents.
- Exact source-location metadata for verification.
- Interactive PDF and notes workspace.
- Reusable study artifacts beyond one chat session.
- Clear separation between core T5 NLP contribution and optional conversational AI.

## Limitations

- Long-document processing can be computationally expensive.
- Hierarchical summarization may introduce information loss across stages.
- ROUGE focuses heavily on textual overlap and cannot fully judge factuality.
- PDF extraction quality depends on the structure and encoding of the input document.
- Language-wise performance may vary across the five dataset languages.
- Personalization features depend on sufficient user interaction data.
- External assistant functionality introduces a dependency when enabled.

### 4.15 Project Planning and Team Roles

| Phase    | Major Work                         | Expected Deliverable                    |
|----------|------------------------------------|-----------------------------------------|
| Phase 1  | Requirements and literature review | Problem definition and design basis     |
| Phase 2  | Application skeleton               | Frontend/backend/database connectivity  |
| Phase 3  | PDF extraction                     | Source text and coordinate records      |
| Phase 4  | Preprocessing                      | Clean chunks and topic segmentation     |
| Phase 5  | T5 summarization                   | Paragraph-level generated summaries     |
| Phase 6  | Hierarchy                          | Topic/page/chapter notes                |
| Phase 7  | Traceability UI                    | Click-to-source and reverse lookup      |
| Phase 8  | Evaluation                         | ROUGE and qualitative analysis          |
| Phase 9  | Advanced features                  | Search, graph, heatmap, assistant, viva |
| Phase 10 | Testing and documentation          | Final report and demonstration          |

The team contribution statement records individual responsibility areas. For final assessment, the contribution percentages and evidence should be aligned with actual Git history, code ownership, testing records and report sections.

#### 4.16 Budget and Resource Considerations

The project is primarily software-based and relies on open-source development libraries and pretrained models. Major cost drivers are computational resources, cloud deployment and any external API usage. During development, local execution reduces recurring infrastructure cost. The planned deployment architecture uses Docker Compose and Render later, while MongoDB Atlas provides cloud-hosted database infrastructure.

| Resource                            | Nature           | Cost Consideration                                           |
|-------------------------------------|------------------|--------------------------------------------------------------|
| Python / open-source libraries      | Software         | No software license cost                                     |
| Hugging Face Transformers / PyTorch | Software         | Open-source; compute is the main cost                        |
| MLSUM/MLSAM dataset                 | Dataset          | Used as project dataset; access conditions should be checked |
| Local development machine           | Hardware         | Existing system                                              |
| MongoDB Atlas                       | Cloud service    | Free/paid tier depending on usage                            |
| Groq API                            | External service | Usage-dependent if enabled                                   |
| Render / hosting                    | Cloud deployment | Usage-dependent if deployed                                  |
| Storage / bandwidth                 | Infrastructure   | Depends on document volume                                   |

# CHAPTER 5

## CONCLUSION & REFLECTION

### 5.1 Conclusion

This capstone project presents an automatic text summarization system based on a T5-family Transformer and extends it into a traceable academic PDF-to-notes platform. The project addresses a practical problem: students often need concise representations of lengthy documents, but generic summaries may be difficult to verify and may not preserve the document's logical structure.

The selected approach uses abstractive generation so that the model can rewrite and condense source information. The MLSUM/MLSAM multilingual dataset provides article–summary pairs for supervised summarization, while ROUGE-1, ROUGE-2 and ROUGE-L provide standard automatic evaluation measures. The system architecture also incorporates PyMuPDF, spaCy, sentence-transformers, vector search and NetworkX to support the wider learning workflow.

The most important engineering decision is the preservation of source metadata. Page numbers, paragraph identifiers and text coordinates are retained from extraction through preprocessing and summarization. This makes it possible for a generated note to act as an entry point back into the original PDF. The design therefore treats trust and verification as part of the product architecture rather than as an afterthought.

The hierarchical paragraph → topic → page → chapter strategy addresses the practical difficulty of summarizing very long documents within transformer context limits. At the same time, it gives students several levels of revision detail. The final system is consequently not just a summarization model, but an integrated document-understanding and learning workflow.

### 5.2 Future Work

- Extend support to additional low-resource and regional languages.
- Evaluate language-specific performance rather than relying only on aggregate metrics.
- Adapt the summarization model to academic, medical, legal and technical domains.
- Investigate long-context transformer models alongside the hierarchical approach.
- Introduce factuality and consistency checks for generated summaries.
- Use human-in-the-loop evaluation with students and subject experts.

- Optimize inference through quantization, pruning, distillation or smaller task-specific models.
- Improve semantic confidence scoring and evidence ranking.
- Develop richer adaptive learning models from reading, rereading, quiz and doubt signals.
- Deploy the system as a scalable web/mobile application.
- Improve offline export and personal revision-notebook capabilities.
- Measure latency, memory use, throughput and cost under realistic document workloads.

### **5.3 Professional Learning & Individual Reflection**

#### **G. Jaswanth (192472235)**

The most difficult engineering problem I encountered was designing a long-document summarization workflow without losing the relationship between generated content and its source. I addressed this by using hierarchical paragraph-to-topic/page/chapter processing while preserving page, paragraph and coordinate metadata throughout the NLP pipeline. A key engineering decision I personally made was to treat source traceability as a core system requirement because academic users need evidence before trusting generated notes. I independently strengthened my knowledge of T5 encoder-decoder models, tokenization, ROUGE evaluation, PDF coordinate extraction and source-grounded NLP. If I repeated the project, I would add stronger factuality evaluation and benchmark the system systematically across document lengths and languages.

#### **Pradeepkumar (192425016)**

The most difficult engineering problem I encountered was preparing multilingual article-summary data in a consistent format suitable for transformer-based learning. I addressed it through structured validation, preprocessing, tokenization and separation of training, validation and test data. A key engineering decision I supported was using a pretrained T5-family model so that transfer learning could be used instead of training a transformer from scratch. I independently acquired deeper knowledge of multilingual summarization datasets, Hugging Face model pipelines and automatic evaluation. If I repeated the project, I would perform more systematic language-wise experiments and hyperparameter comparisons.

#### **Yuvaraj R (192524058)**

The most difficult engineering problem I encountered was integrating the NLP processing pipeline with an interactive application without making the user interface dependent on model internals. I

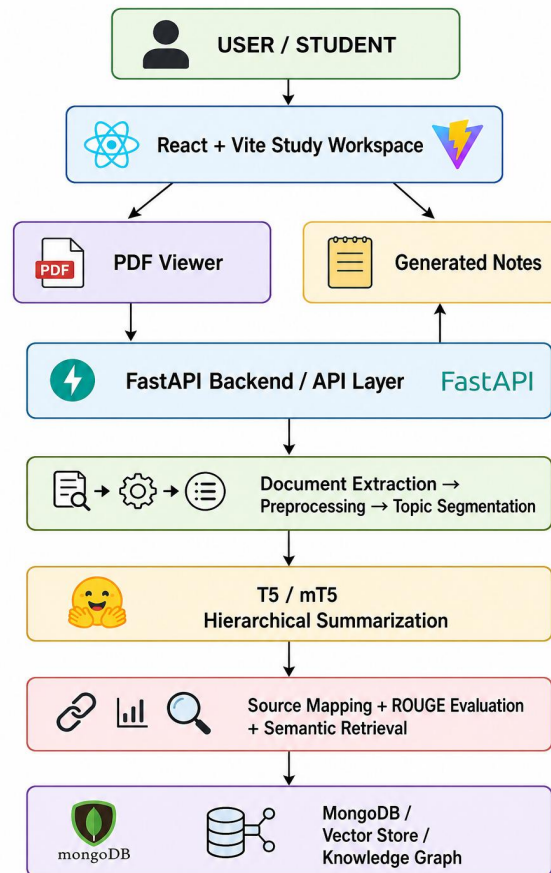
addressed this by separating the frontend, backend, document-processing and summarization responsibilities into modular components. A key engineering decision I supported was using FastAPI as the backend boundary because the NLP pipeline is Python-native and can therefore be integrated without an additional language bridge. I independently developed knowledge of API integration, interactive document workflows and asynchronous processing requirements. If I repeated the project, I would place greater emphasis on deployment testing, response-time monitoring and usability evaluation.

## REFERENCES

1. Raffel, C., et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140), 1–67.
2. Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*.
3. Wang, J., Meng, F., Zheng, D., Liang, Y., Li, Z., Qu, J., & Zhou, J. (2023). Towards Unifying Multi-Lingual and Cross-Lingual Summarization with Many-to-Many Models. *arXiv preprint*.
4. Daraghmi, E., Atwe, L., & Jaber, A. (2025). A Comparative Study of PEGASUS, BART, and T5 for Text Summarization Across Diverse Datasets. *Future Internet*, 17(9).
5. Krishna, R., Somasundaram, K. M., Arulmozhivarman, P., et al. (2025). Deep learning for text summarization using NLP for automated news digest. *Scientific Reports*, 15, 36343.
6. MLSUM / MLSAM multilingual summarization dataset documentation and dataset record used by the project.
7. Hugging Face Transformers documentation for T5-family transformer model and tokenizer implementation.
8. PyTorch documentation for deep-learning model implementation and tensor computation.
9. spaCy documentation for sentence processing and NLP pipeline components.
10. PyMuPDF documentation for PDF text extraction and text-span coordinates.

# APPENDICES

## Appendix A – System Architecture



The architecture follows the project design described in the supplied project overview. The source PDF and generated notes remain connected through page, paragraph and coordinate metadata. The optional conversational assistant is kept outside the core summarization contribution and uses retrieved PDF content as context.

## Appendix B – Detailed NLP Pipeline

| Stage      | Input              | Processing                 | Output             | Evidence                  |
|------------|--------------------|----------------------------|--------------------|---------------------------|
| Extraction | PDF                | Read pages and text spans  | Text + coordinates | Page/paragraph            |
| Cleaning   | Raw extracted text | Remove noise and normalize | Clean text         | Source reference retained |

|              |                       |                               |                          |                    |
|--------------|-----------------------|-------------------------------|--------------------------|--------------------|
| Segmentation | Clean text            | Sentence/topic grouping       | Logical chunks           | Topic IDs          |
| Tokenization | Chunk                 | SentencePiece/model tokenizer | Token IDs                | Chunk ID           |
| Generation   | Token IDs             | T5 encoder-decoder            | Summary text             | Source refs        |
| Hierarchy    | Lower-level summaries | Aggregate and summarize       | Topic/page/chapter notes | Inherited refs     |
| Evaluation   | Generated + reference | ROUGE                         | Scores                   | Test record        |
| Presentation | Notes + evidence      | Render UI and highlights      | Interactive workspace    | Coordinate overlay |

## Appendix C – Sample Data Structure

Illustrative structure for a source chunk:

```
{ document_id, page_number, paragraph_id, text, bbox, topic_id }
```

Illustrative structure for a generated note:

```
{ note_id, level, text, source_refs, topic_id, parent_note_id }
```

Illustrative structure for a learning event:

```
{ student_id, source_id, event_type, timestamp, metadata }
```

## Appendix D – Evaluation Record Template

| Test / Metric          | Dataset / Input         | Configuration             | Observed Value | Remarks                |
|------------------------|-------------------------|---------------------------|----------------|------------------------|
| ROUGE-1                | Test set                | Final model configuration | _____          | Insert actual result   |
| ROUGE-2                | Test set                | Final model configuration | _____          | Insert actual result   |
| ROUGE-L                | Test set                | Final model configuration | _____          | Insert actual result   |
| Average inference time | Representative document | Final environment         | _____          | Insert measured result |

|                      |                         |                   |  |                        |
|----------------------|-------------------------|-------------------|--|------------------------|
| Peak memory          | Representative document | Final environment |  | Insert measured result |
| End-to-end pass rate | Functional tests        | Final build       |  | Insert actual count    |

### Appendix E – Screenshots / Demonstration Evidence

- Project home / document upload screen.
- PDF viewer with extracted content.
- Generated paragraph summary.
- Topic/page/chapter summary hierarchy.
- Click-to-source highlight.
- Reverse lookup from PDF to note.
- ROUGE evaluation output.
- Semantic search result.
- Exam Essentials notebook.
- Knowledge graph with evidence.
- Confusion heatmap.
- Viva simulator.
- Final integrated application workflow.

### Appendix F – Individual Contribution Evidence

| Student       | Suggested Evidence                                                                                          |
|---------------|-------------------------------------------------------------------------------------------------------------|
| G. Jaswanth   | NLP pipeline, source mapping, evaluation, architecture/design work, related commits and report sections.    |
| Yuvaraj R     | Application integration, UI/API work, testing records, related commits and report sections.                 |
| Pradeep Kumar | Dataset preparation, model configuration, training/evaluation records, related commits and report sections. |

### Appendix G – Capstone Outcome Mapping

| Engineering / Professional Area | Report Evidence |
|---------------------------------|-----------------|
| Problem identification          | Chapter 1       |

|                                   |                                     |
|-----------------------------------|-------------------------------------|
| Literature and existing solutions | Chapter 2                           |
| Requirements engineering          | Chapter 3.1                         |
| Design under constraints          | Chapter 3.2–3.10                    |
| Sustainability                    | Chapter 3.11                        |
| Ethics and responsible AI         | Chapter 3.12                        |
| Safety and reliability            | Chapter 3.13                        |
| Security and privacy              | Chapter 3.14                        |
| Risk assessment                   | Chapter 3.15                        |
| Implementation                    | Chapter 4.1–4.8                     |
| Testing and analysis              | Chapter 4.9–4.14                    |
| Project management                | Chapter 4.15–4.16                   |
| Conclusion and future work        | Chapter 5.1–5.2                     |
| Independent learning              | Chapter 5.3                         |
| Team contribution                 | Contribution Statement + Appendix F |
| Technical communication           | Complete report + presentation      |